

Árvore B em Memória Secundária

Implementação de uma classe Árvore B de ordem m operando estritamente em disco

Algoritmos e Estruturas de Dados — AED-PG-2026 (5955001-3)

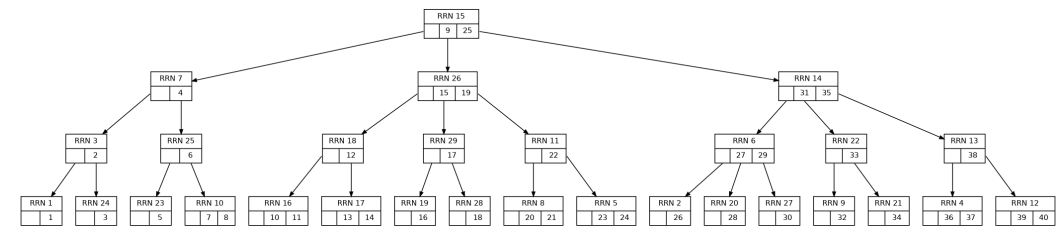
Prof. Dr. José Augusto Baranauskas — DCM / USP

Linguagem: C++17 · 1º Semestre/2026

Aluno: Cássio Takarada (demais integrantes: _____)

2. Decisões de implementação

- Ordem m é constante simbólica de compilação (M): estrutura totalmente parametrizada.
- Raiz da árvore: armazenada no header (registro 0) do arquivo — campo root.
- Um nó por I/O: readNode/writeNode de um registro de `PAGE_SIZE`; a árvore NUNCA é carregada inteira em memória.
- Registro do nó: $n \cdot A[0..m-1]$ (ponteiros RRN) $\cdot K[0..m-1]$ (chaves).
- Reaproveitamento de nós: free list encadeada no próprio arquivo (free_head no header; nó livre marca $n=-1$ e guarda o próximo em $K[1]$).
- allocNode reutiliza a free list antes de estender o arquivo.



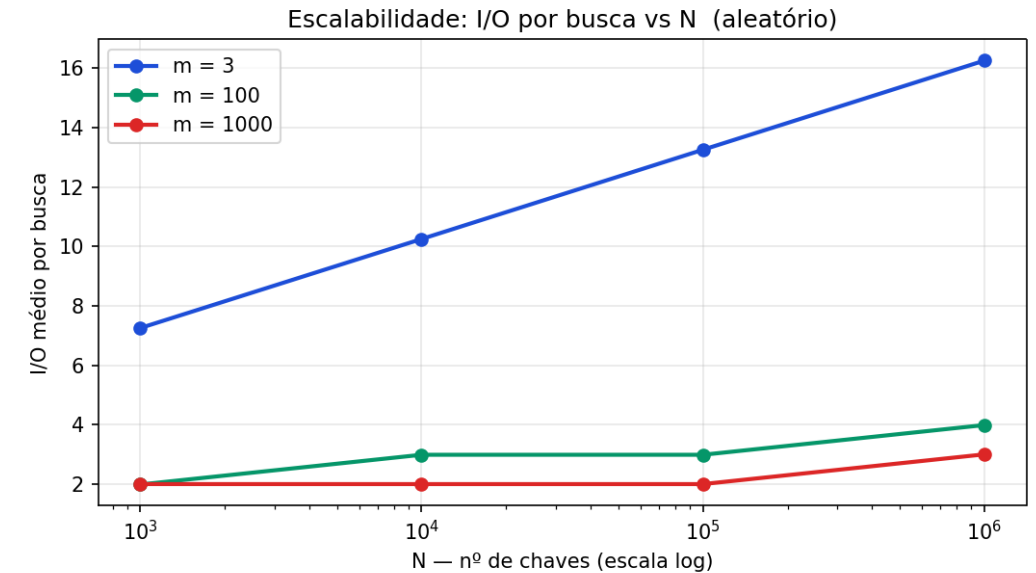
Árvore B ($m=3$) — 40 chaves; cada nó traz seu RRN em disco

3. Métodos implementados

- mSearch / mSearchPath - busca m-way da raiz à folha; retorna (RRN, índice, achou).
- insertB - inserção bottom-up com propagação de split (Equação 1).
- deleteB - remoção com sucessor in-order, redistribuição (empréstimo) e fusão (merge).
- Auxiliares: splitNode, insertInNode, removeFromNode, findSuccessorLeaf.
- Monitoramento: printTree (hierárquico), height, exportDot (Graphviz).
- DiskManager: readNode, writeNode, allocNode, freeNode, contador de acessos, setReuse.
- bench - driver experimental não interativo (CSV de métricas).

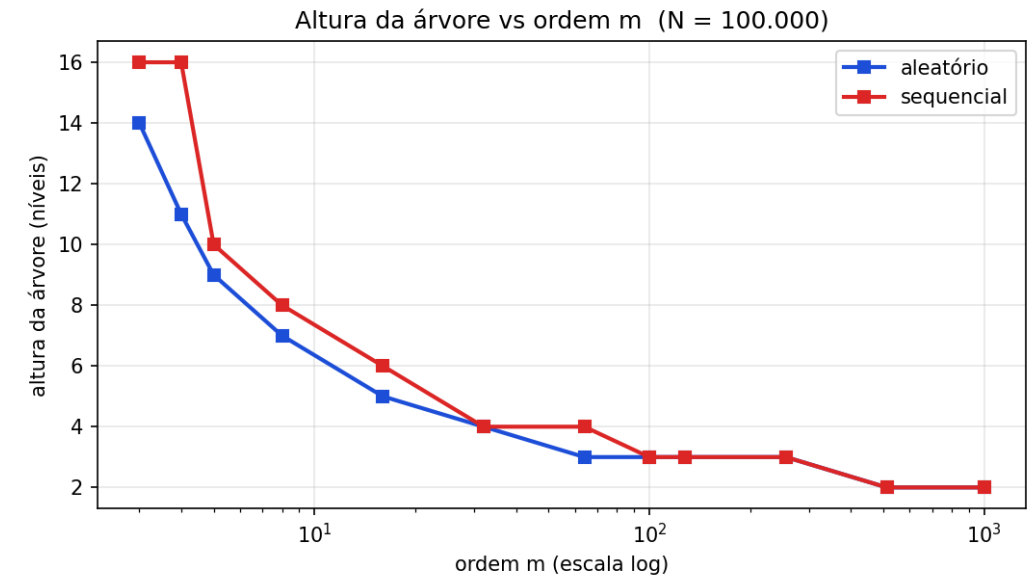
4. Análises efetuadas (experimentos)

- Exp. 1 - Impacto da ordem m: m em {3,4,5,8,16,32,64,100,128,256,512,1000}, $N=10^5$.
- Exp. 2 - Escala do conjunto: N em $\{10^3, 10^4, 10^5, 10^6\}$, m em {3,100,1000}.
- Exp. 3 - Ocupação do arquivo: workload de churn COM e SEM reaproveitamento.
- Exp. 4 - Tempo de execução: CPU (user+sys) vs espera de I/O (getrusage).
- Modos aleatório e sequencial em todos os experimentos.
- Novidade: validação cruzada em 2 máquinas (Notebook x Titan/USP) - métricas de disco idênticas.



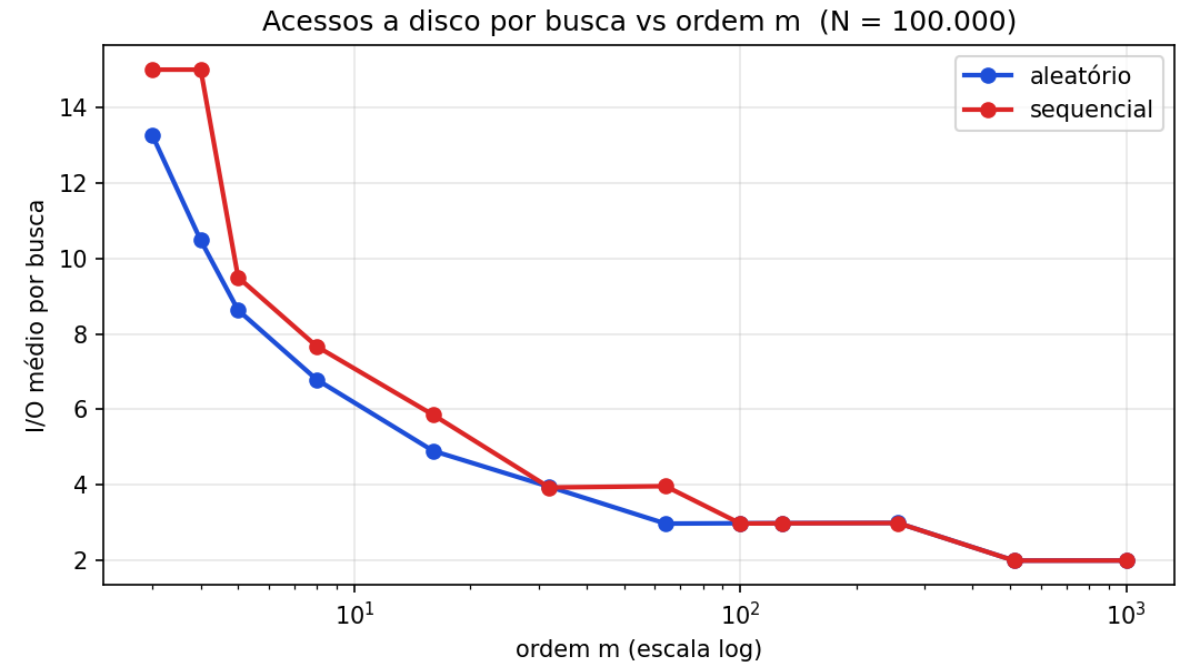
5. Métricas utilizadas

- Acessos ao disco - contador de readNode+writeNode; total e média por operação (a métrica honesta de I/O).
- Altura da árvore - nº de níveis ($\sim \log_m N$).
- Ocupação do arquivo - nós físicos (total_nodes), nós livres (free_nodes) e tamanho em bytes.
- Tempo de parede (wall) - relógio de alta resolução.
- Tempo de CPU - usuário (lógica) e sistema (syscalls de I/O), via getrusage.
- Espera de I/O - wall menos CPU.



6. Tabela de Resultados - impacto de m (N=10^5, aleatório)

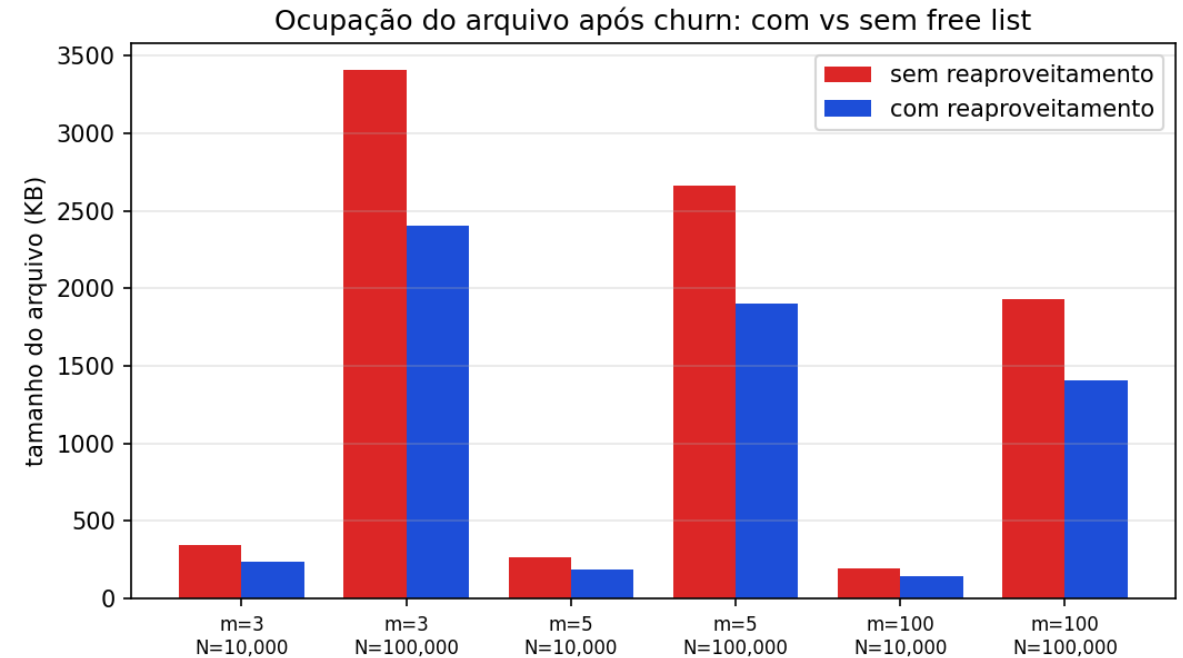
m	altura	busca I/O/op	inser. I/O/op	rem. I/O/op	arquivo (KB)
3	14	13.25	16.90	19.67	2,040
4	11	10.48	13.51	16.76	1,829
5	9	8.63	11.50	13.52	1,589
8	7	6.79	9.05	10.91	1,411
16	5	4.90	7.00	8.16	1,251
32	4	3.95	5.95	6.77	1,189
64	3	2.98	5.04	5.64	1,146
100	3	2.99	4.97	5.51	1,131
128	3	2.99	4.92	5.41	1,121
256	3	2.99	4.57	5.00	1,088
512	2	2.00	4.00	4.41	1,053
1000	2	2.00	3.99	4.34	1,016



I/O por busca cai de 13,25 ($m=3$, altura 14) para 2,00 ($m \geq 512$, altura 2). Ganho satura por volta de $m \sim 64-128$.

6b. Resultados - reaproveitamento de nós (churn)

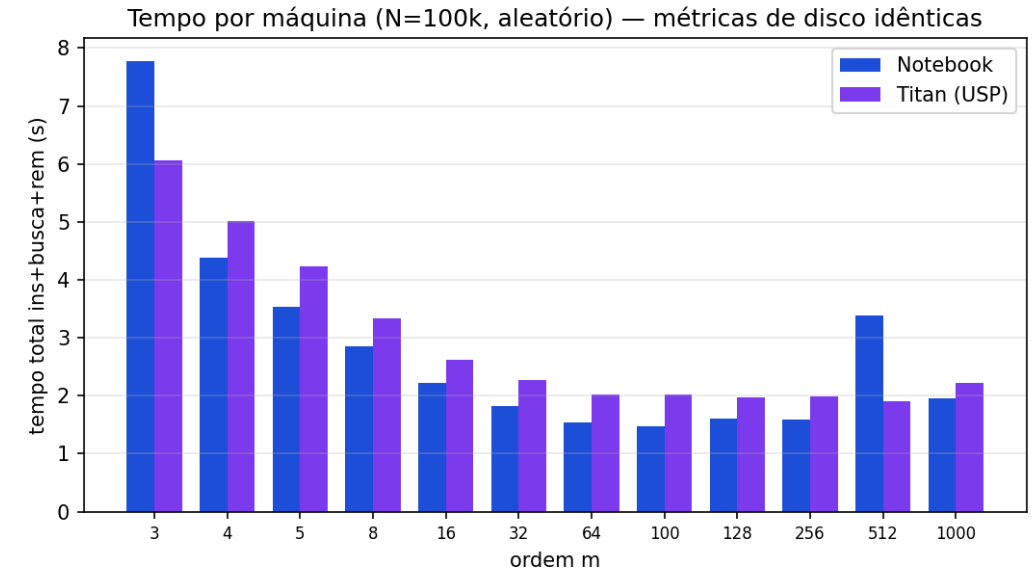
m	N	sem reuso (KB)	com reuso (KB)	economia
3	10,000	342	241	30%
3	100,000	3,407	2,404	29%
5	10,000	266	190	29%
5	100,000	2,663	1,905	28%
100	10,000	196	144	27%
100	100,000	1,931	1,404	27%



O reaproveitamento de nós economiza ~27-30% do tamanho do arquivo neste workload.

7. Utilização de LLM

- Ferramenta: Claude (Anthropic) via Claude Code CLI - auxílio ao desenvolvimento.
- Síntese e discussão dos pseudocódigos dos slides do professor.
- Identificação e correção de 2 bugs: (1) stale-header em insertB; (2) eofbit do fstream em escritas além do fim.
- Geração do harness experimental (bench), tabelas, gráficos e desta apresentação.
- Todo o código foi revisado e validado manualmente pelo autor.



8. Dificuldades · Vantagens x Desvantagens

- Dificuldades: indexação 1-based dos nós; ordem do path no reparo de underflow; eofbit do fstream; garantir 1 nó por I/O (nada em RAM).
- Vantagens da Árvore B: altura baixa => pouquíssimos acessos a disco; sempre balanceada; ideal para memória secundária / SGBDs.
- Desvantagens: nós podem ficar subocupados (~50% no caso sequencial); remoção é complexa; para m muito grande, a busca dentro do nó passa a custar CPU.
- Trade-off central: m maior => menos I/O, mais CPU por nó.

9-10. Aplicações práticas · Conclusão · Referências

- Aplicações: índices de SGBDs (InnoDB/MySQL, PostgreSQL), sistemas de arquivos (NTFS, HFS+, ext4 HTree), key-value stores.
- Conclusão: o I/O por operação cai com m até saturar ($m \sim 64-128$); o reaproveitamento economiza $\sim 27-30\%$ de espaço; resultados determinísticos validados em 2 máquinas.
- Referências:
 - - Comer, D. (1979). The Ubiquitous B-Tree. ACM Computing Surveys.
 - - Knuth, D. The Art of Computer Programming, Vol. 3 - Sorting and Searching.
 - - Folk, M. & Zoellick, B. File Structures.
 - - Baranauskas, J. A. Slides AED-PG-2026 (Árvores B).